Administrivia

- HW4 Progress (Rust Intro)
- Midterm feedback closes today (bonus pt available!)
- Quiz M or W next week

Learning Barrier!

- 1 question on Rust

These two things are NOT equivalent in Rust

```
fn do_stuff(v: &Vec<i32>) {
    v.stuff()
    (*v).stuff()
}
```

Aside: Macros

println! is a macro because it ends with an exclamation mark

Rust functions cannot have a variable number of arguments

Rust macros can!

```
println!(a, b, c) -> io::print(format_args_3(a, b, c))
```

Demo

- Calc (enums)
- Vec Max (Option, Result)

Result<T, E>

- Rust's way of dealing with errors
  - There are no exceptions in Rust! That would break control flow
- std::io::stdin().read_line() returns an Error if the user Ctrl-C'd, for example
- The ? operator lets you unwrap a Result if Ok or return it immediately if Err.

```
match io::stdin().read_line() {
    Ok(line) => {...}
    Err(reason) => {...}
}
```

Rust Standard Library Types

- Option<T> (ft. None and Some(T))
- Result<T, E> (ft. Err(E) and Ok(T))
  - A Result<T, ()> simplifies to an Option<T>
- Vec<T>: a growable vector, items live on the heap
- String: a growable string
- Box<T>: an owned T that lives on the heap
- HashMap<K, V>, Rc<T>, Arc<T>, Mutex<T>

What’s up with slices again?

```
let a: [i32; 3] = [0, 1, 2]; // declare an array on the stack

let b: &[i32] = &a[1..2]; // get a “slice” or a “view” into a.

                          // no copying is done!

let c = b.len(); // Returns 1.

                 // Each slice also contains a length!
```

What’s up with slices again?

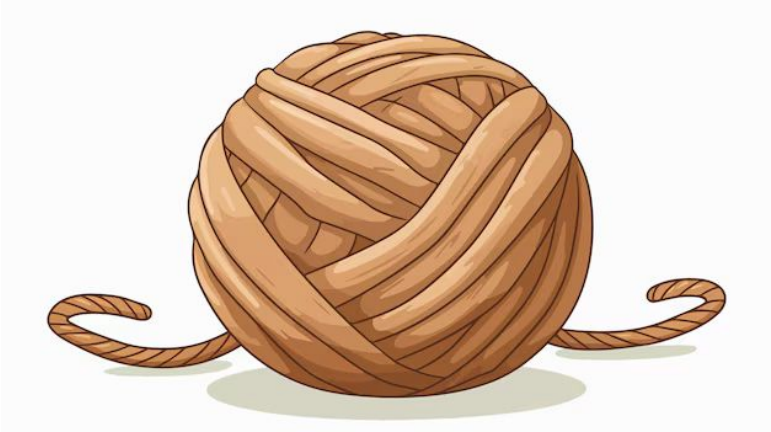
```
fn avg_slice(s: &[f64]) -> f64 {
    let sum: f64 = s.iter().sum();
    sum / (s.len() as f64)
}
```

This input could come from an array or a Vec!

No more passing in a buffer and a length!
s contains both. It is called a “fat pointer”

How do strings work?

- Just like C++’s std::string, the `String` class provides an owned, modifiable string



How do strings work?

```
fn print_s(s: String) {
    println!("s is {s}");
}
fn main() {
    let name = String::from("CSE334");
    print_s(name);

}
```

How do strings work?

```
fn print_s(s: String) {
    println!("s is {s}");
}
fn main() {
    let name = String::from("CSE334");
    print_s(name);
    print_s(name);
}
```

How do strings work?

```
fn print_s(s: &String) {
    println!("s is {s}");
}
fn main() {
    let name = String::from("CSE334");
    print_s(&name);
    print_s(&name);
}
```

How do string literals work?

What is the type of `"CSE334"`?

- `&str`

How do string literals work?

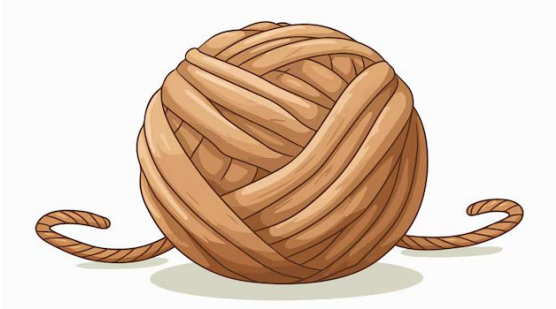
Aah! What is a str?

- A str is an unsized raw sequence of bytes.
  - Because it is unsized, it is impossible to construct
- A &str is a slice into a str that contains a pointer and a length
- When you write a string literal, it’s placed in the read-only section
  - You must get a pointer to it and a length, aka a &str, in order to read it!

String vs &str

String

- Owned
- Modifiable
- On the heap



&str

- Borrow / Slice
- You don’t know where it is

String vs &str

- Strings can be used anywhere a &str is needed
  - Cheap, since we just need a pointer and a length out of the String
- However, creating a String from a &str requires a memcpy

Lifetimes

- Rust doesn’t allow dangling references

```
fn main() {
    let y: &String;
    {
        let x = String::from("hi");
        y = &x;
        // x is dropped
    }
    println!("{}", y);
}
```

Lifetimes

- Rust doesn’t allow dangling references
- The lifetime of &x <= the lifetime of x
- Lifetimes can be explicitly annotated:

```
let name: &'static str = “CSE493L”;
```

Lifetimes (broken)

```
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}
```

What if x lives longer than y?
The compiler has no idea how long the return value's lifetime should be!

Lifetimes (fixed)

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}
```

What happens when we use longest?

```
fn main() {
    let mut a: String = String::from("foo");
    let b: &str = "foobar";
    let longest: &str = longest(&a, b);
    println!("longest is {longest}");
}
```

What happens when we use longest?

```
fn main() {
    let mut a                                  rom("foo");
    let b: &str = "bar";
    let longest: &str = longest(&a, b);
    a.push('!');
    println!("longest is {longest}");
}
```

These two lifetimes are now one!

Mutable reference to a

Immutable reference to a

Compiler error!

```
error[E0502]: cannot borrow `a` as mutable because it is also borrowed
as immutable

42 |     let longest: &str = longest(&a, b);

   |                                -- immutable borrow occurs here

43 |     a.push('c');

   |     ^^^^^^^^^^^ mutable borrow occurs here

44 |     println!("longest is {longest}");

   |                         --------- immutable borrow later used
here
```

What do we do if we really want this behavior?

- We might want longest to return an *owned* string, aka. a String, rather than a &str
- We might not! We can always rely on the caller to create a String if needed.

Traits

Define a set of methods that a type must implement

Like an interface in Java

Trait example

```
trait Animal {
    fn new(name: String) -> Self;
    fn say_hi(&self);
}
```

```
struct Cow {
    name: String,
}
```

```
impl Animal for Cow {
    fn new(name: String) -> Self {
        Cow {name}
    }
    fn say_hi(&self) {
        println!("cow named {self.name}");
    }
}
```

Trait example

```
trait Animal {
    fn new(name: String) -> Self;
    fn say_hi(&self);
}
```

```
struct Dog {
    name: String,
}
```

```
impl Animal for Dog {
    fn new(name: String) -> Self {
        Dog {name}
    }
    fn say_hi(&self) {
        println!("dog named {self.name}");
    }
}
```

Trait example (bad)

```
fn introduce<T>(animal: T) {
    animal.say_hi();
}
```

Compiler error!
Not all Ts define .say_hi()!

Trait example (good)

```
fn introduce<T: Animal>(animal: T) {
    animal.say_hi();
}
```

Trait bound means that this function is only parameterized for T is an Animal

Trait Syntactic Sugar

```
fn introduce(animal: impl Animal) {
    animal.say_hi();
}
```

Complex trait bounds: the where clause

```
fn add_stuff<T, T2>(a: T, b: T) -> T2
where
    T: std::ops::Add<Output = T2> + Clone + Copy + Default,
    T2: std::cmp::PartialEq + From<f64> + Into<f64>,
{
   <function body>
}
```

Associated Types

Traits can define types that the implementer must override.

```
pub trait Animal {
    type Food;
    fn eat(&mut self, food: Self::Food);
}
impl Animal for Cow {
    type Food = Grass;
    fn eat(&mut self, food: Grass) {...}
}
```

Common Traits

- `T: Clone` means that `t.clone() -> T` is defined
- `T: Default` means that `T::default() -> T` is defined
- `T: ToString` means that `t.to_string() -> String` is defined
- `T: From<T2>` means that `T::from(t2: T2) -> T` is defined
  - It *implies* that `T2: Into<T>`, aka `T2::into(t: T) -> T` is defined "automatically"
- `T: Deref` overrides the unary * (dereference) operator
  - Box<T> behaves the same as &T
- `T: Display` means that `t.fmt()` is defined
  - `println!("{t}")` only works on `t: T` where `T: Display`!
- `T: Debug` is similar to display but is auto-generatable for many types
  - `println!("{t:?}")` is the syntax for debug printing `t`

Drop Trait

`T: Drop` means that `t.drop()` is defined.

The compiler calls this for you when a variable goes out of scope.

- Drop == running a destructor on the data
- You can manually drop a variable by using this function: (defined in std::mem)

```
pub fn drop<T>(_x: T) {}
```

Marker Traits

- `T: Copy` means that a copy of `t` can be made via a simple bit-level memcpy
  - No functions need to be implemented to be Copy!
  - Instead, Rust will check to make sure that all elements of `T` are also `Copy`
- `Sized`
  - The type has a known size at compile time (unlike str and dyn T)
- `Send, Sync`
  - The foundation for Rust's "fearless concurrency"
  - We will talk more about these marker traits later!

Derive

Many traits can be automatically derived (when possible), e.g.

```
#[derive(Debug, Copy, Clone)]
struct Foo {
    bar: u64,
    baz: i32,
}
```

Iterators

Let’s put it all together with iterators!

`Iterator`: a trait that defines

```
fn next(&mut self) -> Option<Self::Item>;
```

Where `Self::Item` is an associated type for the `Iterator` trait.

When iterator is implemented for a type, Rust allows you for loop over it!

Iterators

Efficiently implement functional-style programming:

```
let mut list = [
    Rectangle { width: 10, height: 1 },
    Rectangle { width: 3, height: 5 },
    Rectangle { width: 7, height: 12 },
];

list.sort_by_key(|r| r.width);
```

This is a *closure*